

12/4/2020

Manual Desarrollador
Grupo de Investigación GIIRA



ERGOSENT



LEAP
MOTION



UNIVERSIDAD DISTRITAL



CONTROL DE CAMBIOS

VERSIÓN	FECHA	RESPONSABLE	DESCRIPCIÓN
1.0	12 de abril de 2020	Grupo GIIRA	Versión inicial

CAPÍTULO I – TECNOLOGÍAS Y VERSIONES EN USO



1. Unity – Version 2019.3.9f1

Unity es el motor gráfico gratuito más utilizado para crear videojuegos, capaz de crear experiencias para todo tipo de plataforma o consola, desde RPG's hasta shooters. Unity está disponible como plataforma de desarrollo para Microsoft Windows, Mac OS, Linux.



2. Leap Motion – Unity Core Assets Version 4.4.0



El Leap Motion Controller es un módulo óptico de seguimiento de manos que no requiere contacto y captura los movimientos de las manos con una precisión muy alta.

3. Azure Blob Storage

El almacenamiento de blobs es una característica de Microsoft Azure que permite a los desarrolladores almacenar datos no estructurados en la plataforma en la nube de Microsoft. Los blobs se agrupan en "contenedores" que están vinculados a las cuentas de usuario.

Microsoft Azure
Blob Storage



4. Power BI – Version 2.74.5619.862



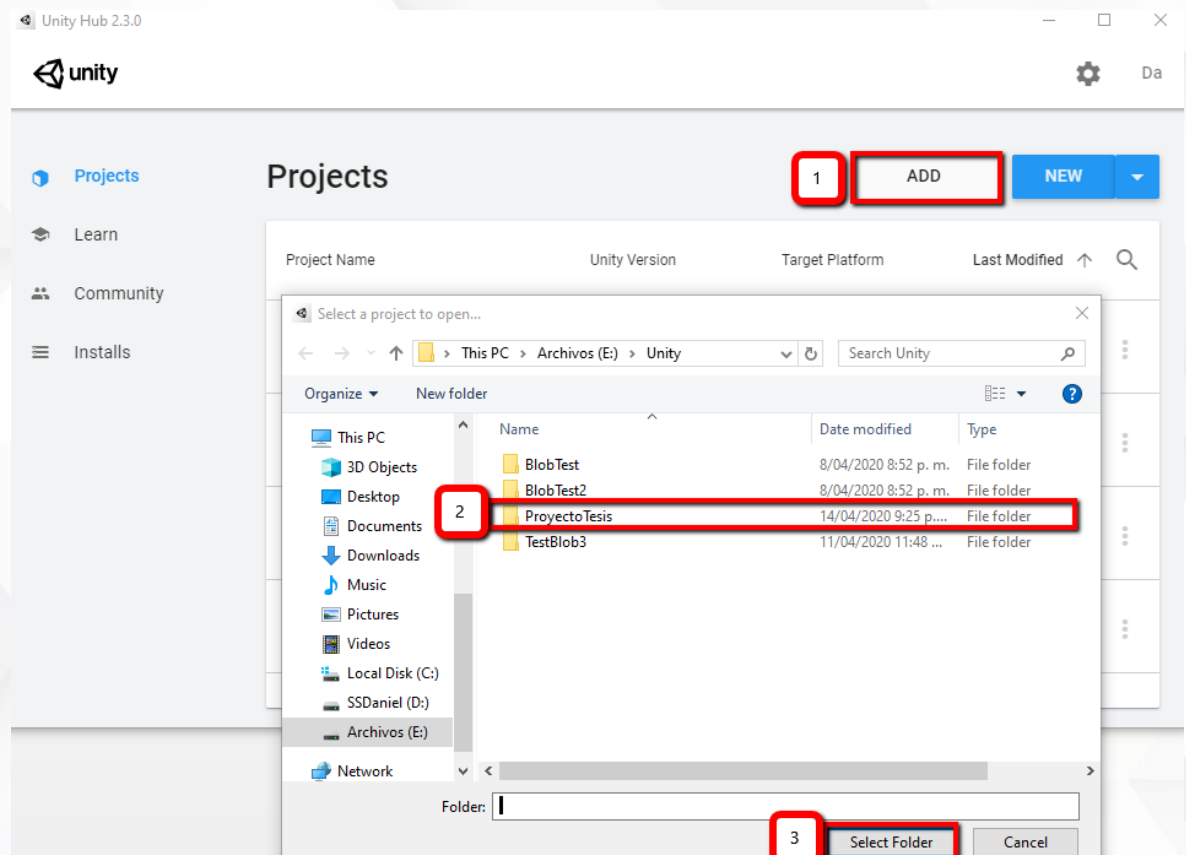
Power BI es un servicio de análisis empresarial de Microsoft, su objetivo es proporcionar visualizaciones interactivas y capacidades de inteligencia empresarial con una interfaz lo suficientemente simple como para que los usuarios finales creen sus propios informes y paneles

5. Cargar el proyecto de Unity

- a. Instalar Unity Hub
- b. Cargar el proyecto



ERGOSENT



c. Iniciar el editor de Unity

CAPÍTULO II – ARQUITECTURA DEL SISTEMA





ERGOSENT

CONCEPTOS GENERALES

Datos de entrada

Los datos de entrada provienen de clases preestablecidas por el API de Leap Motion, a continuación se realizará un barrido de los componentes que proveen esta información y sus características.

LeapProvider y Frame

LeapProvider proporciona datos de objetos de tipo Frame a la aplicación Unity disparando eventos tan pronto como los datos Frame están disponibles. Estos Frame contienen todas las manos rastreadas actualmente a la vista del controlador Leap Motion

Hand, Arm, Finger

La clase Hand (Mano) informa las características físicas de una mano detectada y adicionalmente contiene un atributo de tipo Arm (Brazo). Si el brazo no está completamente a la vista, los atributos del brazo se estiman en función de los atributos de las entidades que se ven combinadas con la anatomía humana típica. La clase Finger (Dedo) representa un dedo rastreado y contiene múltiples instancias de la clase Bone (Hueso).



Bone

La clase Bone proporciona propiedades y métodos que permiten determinar fácilmente el tipo anatómico, la longitud, el ancho, el vector, entre otras características, de un hueso determinado. Estos huesos se ordenan desde la base de la muñeca hasta las puntas de los dedos (índice 0 a 3) y se pueden acceder por medio de sus nombres anatómicos

PrevJoint y NextJoint

Cada hueso cuenta con dos articulaciones, prevJoint (previous joint o articulación previa) que es la base del hueso, más cercana a la muñeca. En términos anatómicos, este es el extremo proximal del hueso. A su vez, nextJoint (articulación próxima) es el final del hueso, más cercano a la punta del dedo. En términos anatómicos, este es el extremo distal del hueso. Estas articulaciones se encuentran representadas en forma de Vector tridimensional.

CLASES DEL APLICATIVO



Nombre: Incident (*Incidente*)

Descripción: Clase modelo que contiene las propiedades de los incidentes (eventos que ocurren cuando las normas ergonómicas fallan)



Nombre: DataStorage (*Almacenamiento de datos*)

Descripción: Se encarga de realizar todos los comandos sobre la base de datos



Nombre: LeapOrchestrator (*Orquestador Leap*)

Descripción: Hereda del componente LeapProvider nativo del LeapMotionSDK, se encarga de llamar a todas las demás clases, ejecutar el motor de procesamiento y comunicar los eventos entre los mismos



Nombre: CSVManager (*Gestor de CSV*)

Descripción: Administra la exportación de los datos en la base de datos a un archivo CSV



Nombre: BlobManager (*Gestor de Blob*)

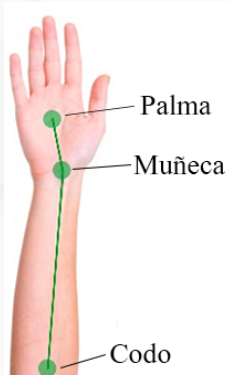
Descripción: Se encarga de cargar el CSV generado por la clase CSVManager al blob de Azure

MODULO RECOLECCIÓN DE DATOS



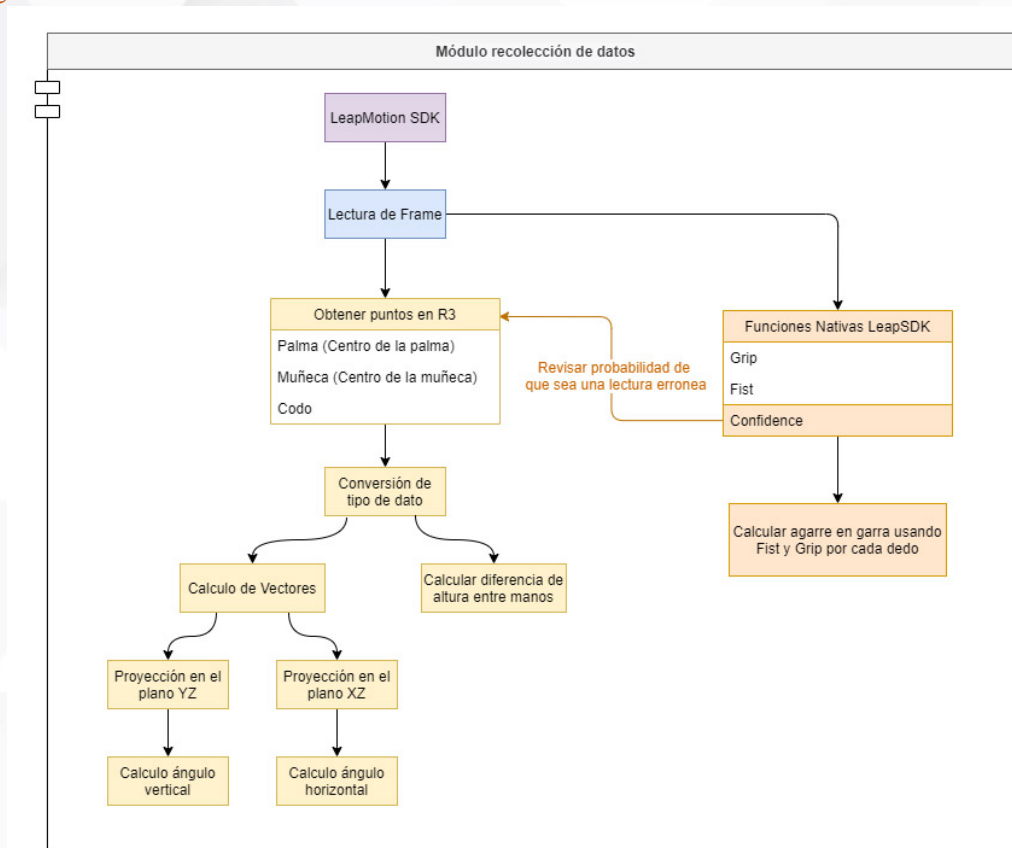
ERGOSENT

Puntos leídos por el software



Se toman estos puntos porque la ausencia de articulaciones entre el codo y la muñeca permite establecer un vector que sirve como referente fijo frente al vector generado entre la muñeca y la palma, de esta manera facilitando el cálculo de los ángulos. Adicionalmente a esto, se utilizan las funciones Grip, Fist y Confidence (Nativas de LeapMotion), que indican si la mano esta en forma de agarre natural o de puño respectivamente, con esta información y el modelo vectorial planteado, se pueden resolver todas las reglas ergonómicas a evaluar.

Diagrama módulo recolección de datos



MODULO CLASIFICACIÓN DE DATOS



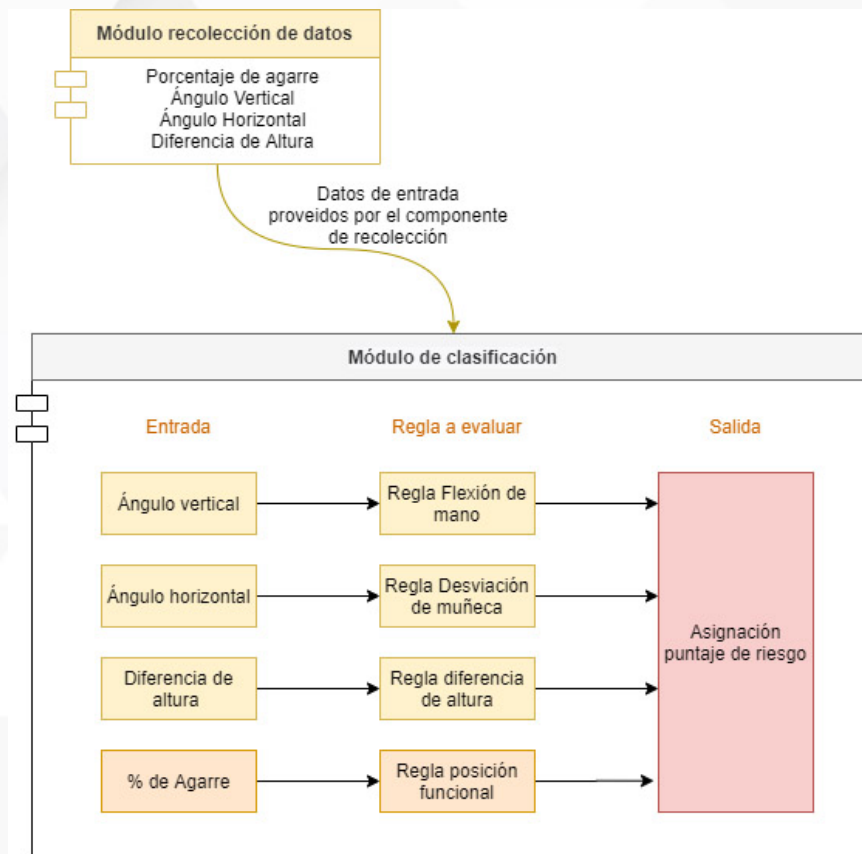
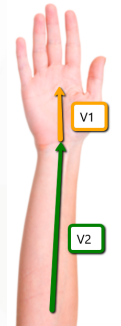
ERGOSENT

Los vectores v_1 y v_2 y el cálculo de los ángulos

Para comenzar, se toman los tres puntos de referencia: palma, muñeca y codo, posteriormente se proyectan en el eje XZ (para obtener el ángulo horizontal) o en el eje XY (para obtener el ángulo vertical) y se restan los puntos palma-muñeca para obtener el vector v_1 y los puntos muñeca-codo para obtener el vector v_2

Así, $\theta = \cos^{-1} \frac{v_1 \cdot v_2}{|v_1||v_2|}$ donde θ es el ángulo entre los vectores v_1 y v_2 , luego se convierte el ángulo θ a grados utilizando la constante $\frac{180}{\pi}$

Finalmente, teniendo el ángulo θ que representa el ángulo vertical u horizontal dependiendo de la proyección realizada se compara con los ángulos de las reglas ergonómicas





ERGOSENT

```
protected float CalcularAnguloHorizontal(Hand hand)
{
    /* Regla 1: Desviación Horizontal (palma - muñeca - codo)
    * Arm.NextJoint - Arm.PrevJoint - hand.PalmPosition
    *
    * DEF: Utiliza los valores centrales de la mano, la muñeca y la palma
    * Puede fallar porque los valores del codo son estimados cuando el sensor no ve
    * el codo. Además no reconoce los casos en los que el codo está hacia abajo, solo arriba
    */

    Arm arm = hand.Arm;
    int ajustePosicionFuncional;

    //Calculo del angulo horizontal
    //Obtener los 3 puntos de análisis sin su posición en Y (Proyección en el plano)
    Vector3 palmaSinY = hand.PalmPosition.ToVector3();
    Vector3 muñecaSinY = arm.NextJoint.ToVector3();
    Vector3 codoSinY = arm.PrevJoint.ToVector3();

    palmaSinY.Set(palmaSinY.x, 0, palmaSinY.z);
    muñecaSinY.Set(muñecaSinY.x, 0, muñecaSinY.z);
    codoSinY.Set(codoSinY.x, 0, codoSinY.z);

    //Restar los vectores
    Vector3 palma_muñeca_SinY = palmaSinY - muñecaSinY;
    Vector3 muñeca_codo_SinY = muñecaSinY - codoSinY;

    //Hallar el producto punto
    float productoPuntoSinY = Vector3.Dot(palma_muñeca_SinY, muñeca_codo_SinY);

    //Hallar magnitud de los vectores
    float mag_palma_muñeca_SinY = Vector3.Magnitude(palma_muñeca_SinY);
    float mag_muñeca_codo_SinY = Vector3.Magnitude(muñeca_codo_SinY);

    //Calculo de la direccion de la mano (es necesario porque el angulo viene sin signo)
    // -1 para izquierda / 1 para derecha
    float direccionMuñeca = Mathf.Sign(palmaSinY.x - muñecaSinY.x + 6) * -1;
    //16

    //Ajustar la desviacion cubital
    //Por definición la desviación cubital para una posición funcional son diez grados
    //el ajuste causa que el cero se encuentre al hacer una ligera desviación.
    if (hand.IsLeft){ajustePosicionFuncional = 10;}else{ajustePosicionFuncional = -10;}

    //Obtener coseno del angulo
    float cosAnguloSinY = productoPuntoSinY / (mag_palma_muñeca_SinY * mag_muñeca_codo_SinY);
    //Obtener el inverso del coseno, convertir a grados y agregar ajuste
    float anguloHorizontal = Mathf.Round((Mathf.Abs(Mathf.Acos(cosAnguloSinY) * Mathf.Rad2Deg) *
    direccionMuñeca) + ajustePosicionFuncional);

    return anguloHorizontal;
}
```

DISEÑO MODELO ARQUITECTURAL

El SDK proveído por LeapMotion funciona a partir de servicios que son consumidos por el desarrollador dependiendo de la información que se quiera obtener. Para este proyecto, la clase a que tiene los parametros de entrada requeridos es la LeapServiceProvider, esta clase proporciona objetos {Frame que contienen manos.

Es decir, cualquier clase que necesite datos de la mano del sensor necesitará una referencia a un LeapServiceProvider para obtenerlos. En conclusión, la arquitectura del software debe implementar como componente principal (fuente de eventos) este servicio.

Componentes de EDA en ErgoSent

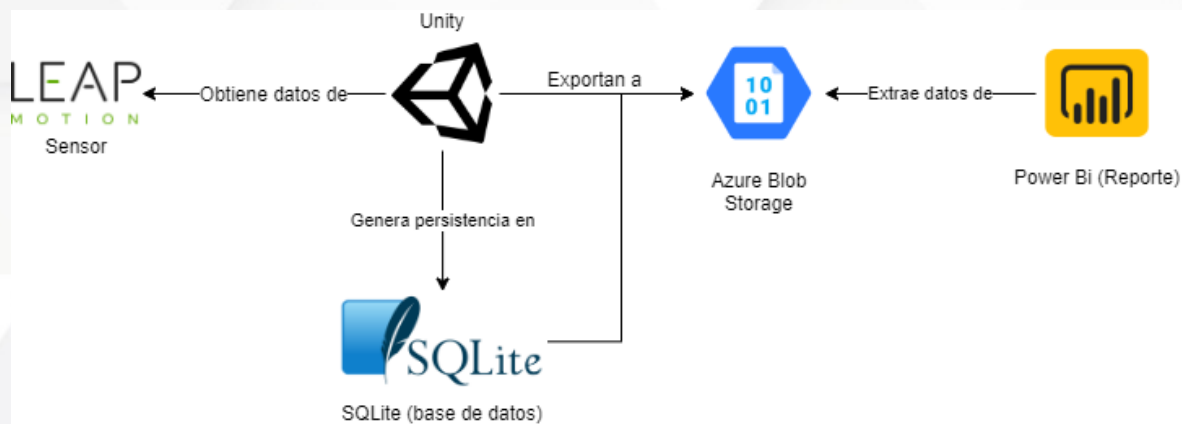
Para este proyecto, los componentes de EDA son representados por las entidades presentadas en la figura mostrada a continuación, donde los componentes de la parte superior son administrados por el motor de procesamiento de eventos, y los demás recursos (Persistencia, Servicios y Eventos) son integrados por el orquestador.





ERGOSENT

En resumen, Unity obtiene datos del sensor LeapMotion para luego ejecutar los procesos de negocio establecidos y finalmente realizar persistencia de las posturas de riesgo en la base de datos. Finalmente, PowerBi consume los datos depositados en la base de datos para generar el reporte de evaluación de puestos de trabajo. Esta interacción se puede observar en la figura a continuación



Metadatos o especificaciones de los eventos: Las especificaciones de los eventos y reglas de procesamiento están dadas e implementadas por los servicios nativos del SDK Leapmotion. Para el aplicativo, estos eventos son construidos por el servicio LeapServiceProvider.

Motor de procesamiento de eventos: Se implementará un motor simple de procesamiento, codificado nativamente sin utilización de librerías de terceros. Los datos de los eventos acontecidos se guardarán en persistencia para efectos de auditoría y posibles análisis históricos. El motor simple de dato será la clase. El procesamiento de eventos será implementado en la clase XRDataProvider y la persistencia en la clase DataStorage

Integración Empresarial: Para el proyecto se creará una clase gestora llamada OrquestadorLeap que servirá como orquestador del sistema. Es el encargado de coordinar todos los componentes que se muestran en la figura anterior

Fuentes y objetivos: Como fuentes tenemos el dispositivo LeapMotion visto a través del flujo de eventos del servicio LeapServiceProvider, como objetivos se tiene la persistencia en SQLite a través de la clase DataStorage.

Flujo del proceso de negocio

En la figura mostrada a continuación, se muestra el flujo del proceso para ErgoSent. Cuando se detecta una mano dentro del espacio de captura del sensor LeapMotion y esta mano es válida, se genera inmediatamente un evento que contiene un Frame a través de la clase LeapServiceProvider. El evento Frame Read es puesto en el Canal de eventos y recibido por el motor de procesamiento de



ERGOSENT

eventos simple. Las reglas de procesamiento para este tipo de evento (Frame read) ordena dos instrucciones. Invocar el servicio evaluar riesgo postural para determinar si la posición leída es ergonómica, y de ser pertinente la publicación del evento para sus suscriptores. Los suscriptores son el gestor de base de datos y el aplicativo de Unity.

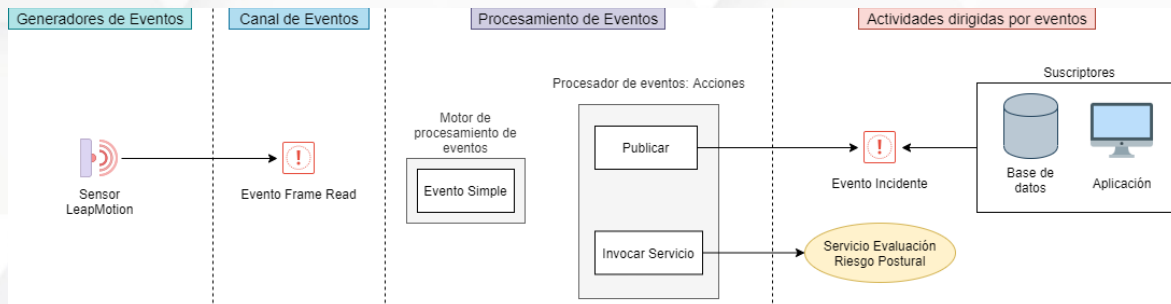


Diagrama de flujo del motor simple de procesamiento (Módulo de interpretación)

El diagrama de flujo ilustrado en la figura a continuación comienza cuando se inicia el aplicativo al comienzo de la jornada laboral del trabajador, posteriormente se inicializa la base de datos, y se recibe el primer frame de datos desde el sensor. A este frame de datos se le extrae la matriz de puntos capturados y se procede a calcular las métricas ergonómicas asociadas según lo establecido por el modulo de clasificación de datos. Una vez calculadas, se evalúan para ver si estas corresponden a un riesgo postural de acuerdo a lo establecido en las normas internacionales y dependiendo del resultado de divide el flujo proceso en dos.

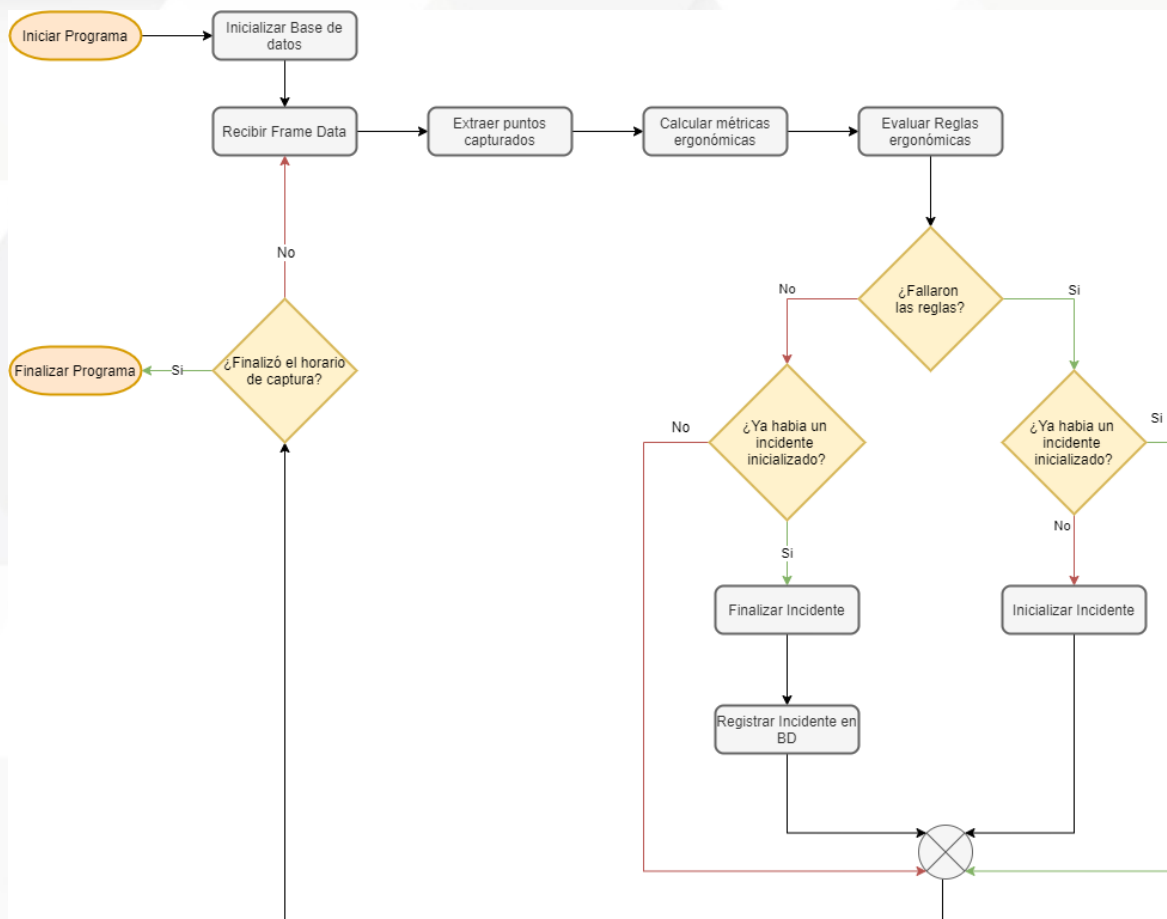
Si la respuesta es afirmativa y existe un riesgo postural, se debe revisar si ya había un incidente iniciado, es decir, si ya existía un riesgo postural en el anterior frame de datos recibido, en caso afirmativo simplemente se continuará el flujo, en caso negativo, se inicializará el incidente, tomando la hora y fecha en la cual ocurrió el incidente, la regla postural que falló y su puntaje de riesgo postural asociado, luego, continúa el flujo.

Si la respuesta es negativa y no existe un riesgo postural, se debe revisar si ya había un incidente iniciado, en caso afirmativo, quiere decir que se estaba asumiendo una postura no ergonómica, pero el miembro ya regresó a una posición neutra, entonces, se debe finalizar el incidente, de esta manera calculando la duración del evento, el grado promedio de desviación desde la posición funcional de la mano y finalmente registrando esta información en la base de datos.



ERGOSENT

Continuando con el flujo de datos, antes de recibir un nuevo frame de datos, se debe validar si ya se ha cumplido la jornada de captura que se quiere realizar, por ejemplo, si el profesional estipuló realizar un seguimiento por 3 horas, verificar si ya transcurrió el tiempo establecido. En caso afirmativo, se finalizan los incidentes que se encuentren abiertos y se finaliza el programa, en caso negativo, se continúa y comienza el ciclo de nuevo al recibir el siguiente frame de información

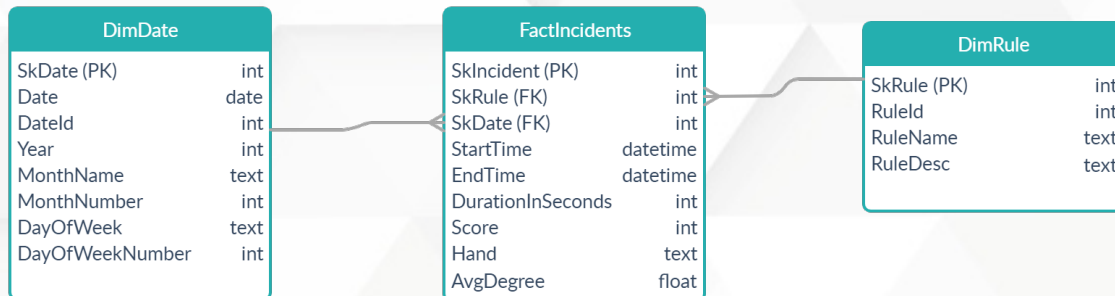


Modelo de datos en Unity

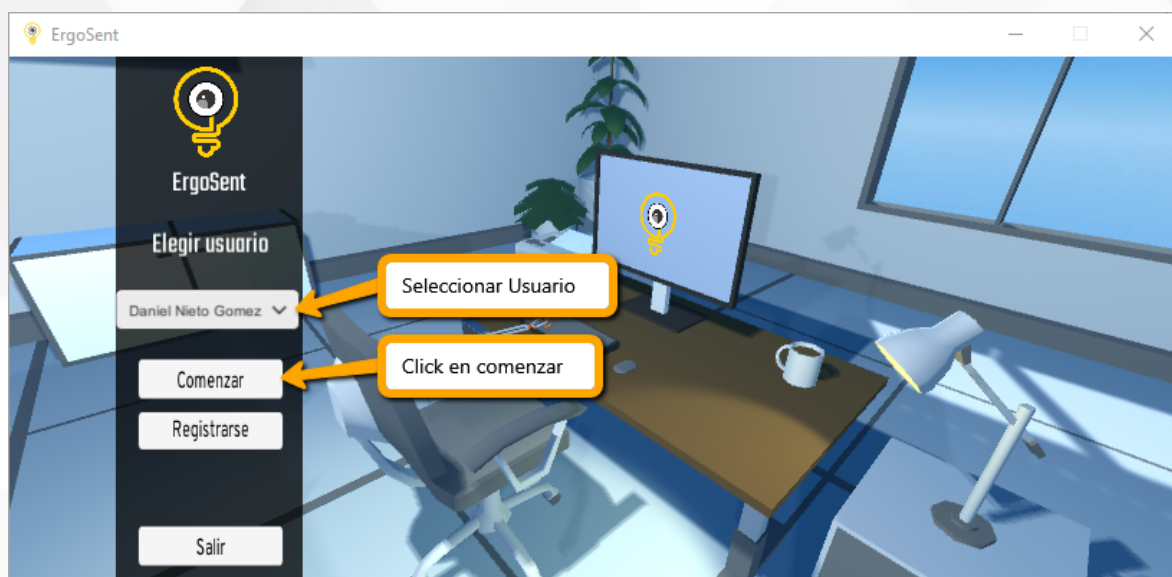
Con el objetivo de construir un modelo de datos consistente, se tuvieron varias reuniones con la Product Owner que ayudaron a comprender las variables de análisis más importantes. De acuerdo a esto, se planteó un modelo de datos desnormalizado de tal manera que se facilite la extracción, transformación y carga de datos a la estructura de análisis de PowerBi.



ERGOSENT

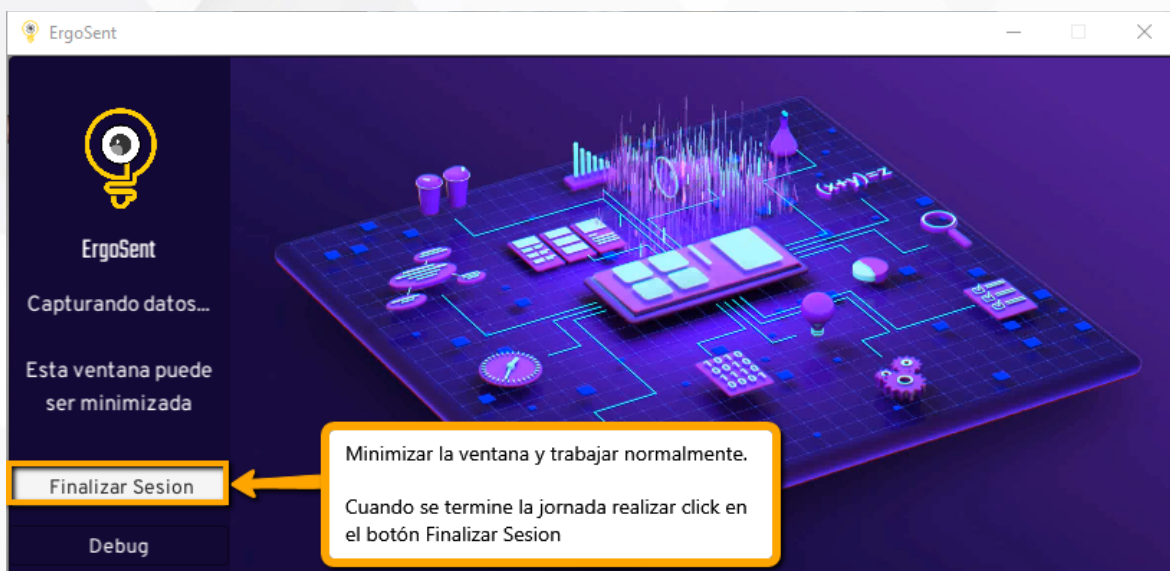
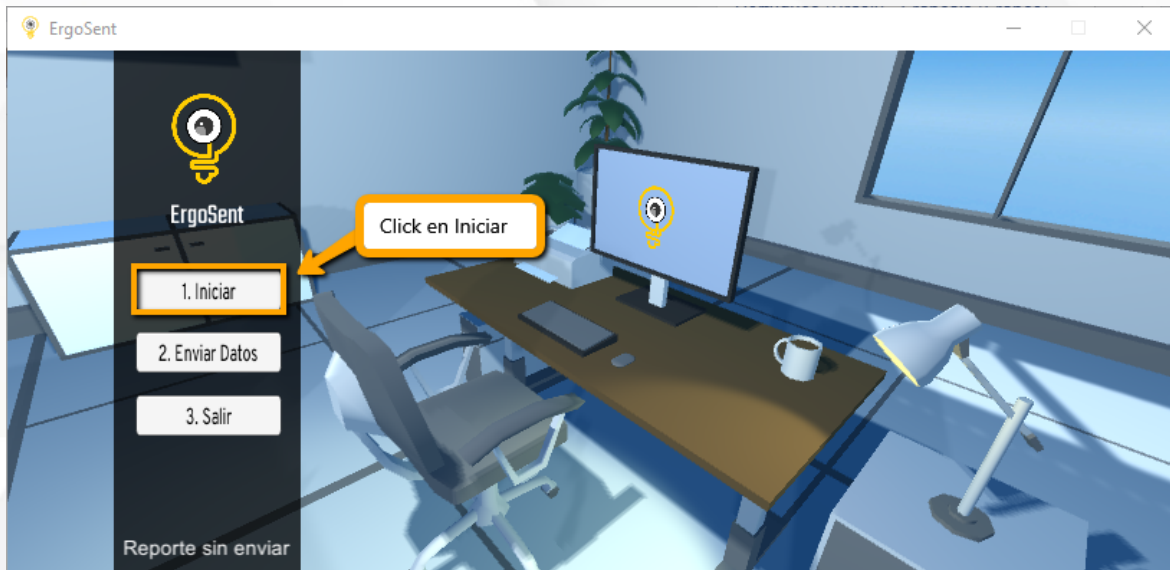


Interfaz de la aplicación





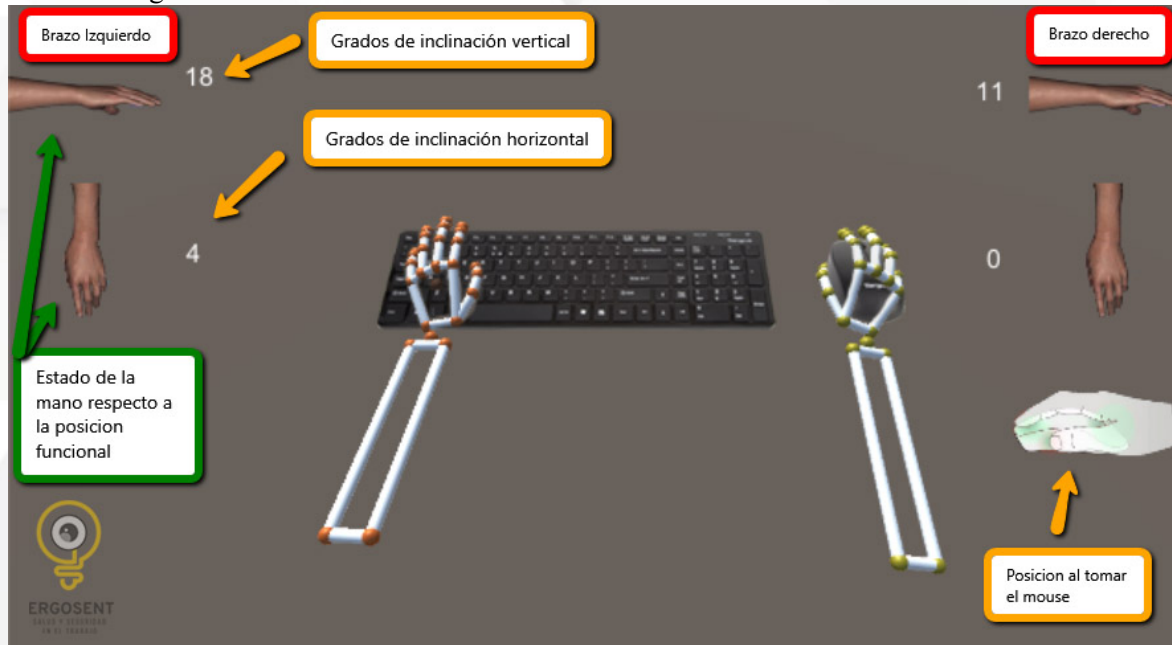
ERGOSENT



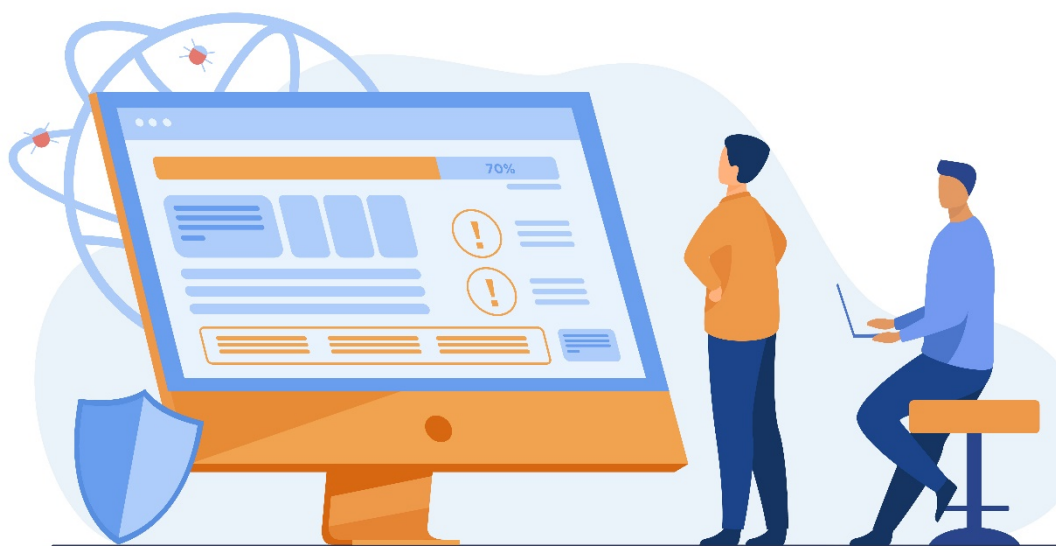


ERGOSSENT

Modo Debug:



CAPITULO III - DESPLIEGUE DE LA APLICACIÓN

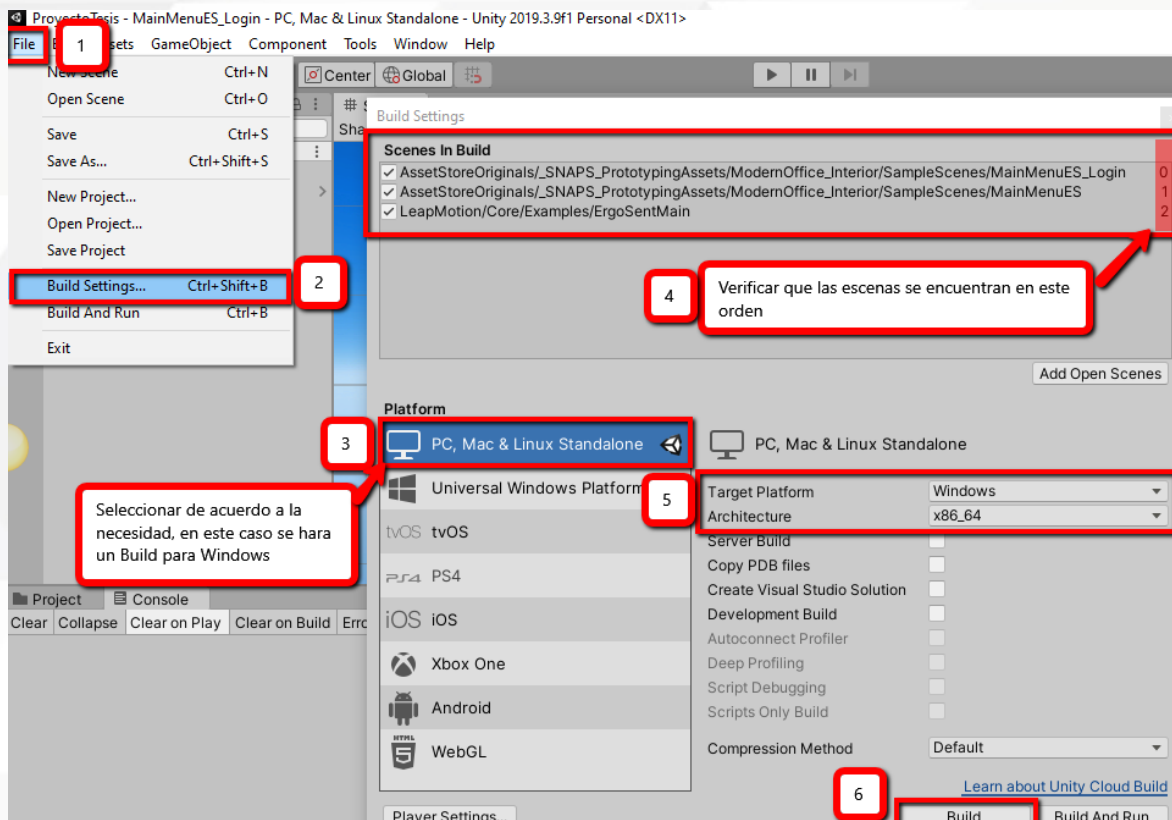




ERGOSENT

Despliegue en Unity.

Para realizar el despliegue del aplicativo se debe hacer click en File -> Build Settings, luego seleccionar de acuerdo a la necesidad la plataforma objetivo, en este caso es PC, Mac & Linux Standalone. Luego, se debe verificar que las escenas se encuentran en el orden de presentación (Login -> Menu -> Main), posteriormente, se selecciona Windows como plataforma objetivo en arquitectura 64 bits. Finalmente, se hace click en Build para generar el ejecutable del aplicativo.



Al finalizar el proceso, se deben encontrar los siguientes archivos en la carpeta objetivo:



ERGOSENT

Name	Date modified	Type	Size
ErgoSent_Data	15/04/2020 6:46 p. m.	File folder	
MonoBleedingEdge	15/04/2020 6:46 p. m.	File folder	
ErgoSent.exe	15/04/2020 6:46 p. m.	Application	636 KB
UnityCrashHandler64.exe	6/04/2020 11:19 p. m.	Application	1.069 KB
UnityPlayer.dll	6/04/2020 11:19 p. m.	Application exten...	25.218 KB

Para copiar el aplicativo en otro medio, debe copiarse toda la carpeta contenedora, no solo el ejecutable